

Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático/a: Erick Marroquín



Ejercicio 1 – Overloading y relaciones entre clases. Análisis y Diseño.

Nombre: Javier Alejandro Apen Solis

Carné: 26864

31/07/2026

Contenido

Análisis	3
1. Identificación de clases	3
2. Tabla de propiedades	3
3. Tabla de métodos	4
¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)?.....	5
¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?	6
Diseño	6
Diagrama de clases - UML	6
Justificaciones de asociaciones	7

Análisis

1. Identificación de clases

Clase	Rol MVC	Por qué existe
Comprador	Model	Datos de quien quiere el boleto.
Localidad	Model	Estado y reglas de una sección.
Ticket	Model	Algoritmo para la solicitud de ticket.
GestorVentas	Controller	Calculo de reportes y validaciones.
Menu	View	Muestra menus, ingreso de datos e imprime resultados.
Main	Driver	El main().

2. Tabla de propiedades

Clase	Propiedad	Tipo de dato	Modificador de acceso	Propósito
Comprador	nombre	String	private	Identificar al comprador.
Comprador	email	String	private	Correo del comprador.
Comprador	cantidadBoletos	int	private	Numero de boletos deseados.
Comprador	presupuestoMax	double	private	Límite de gasto del comprador.
Localidad	nombre	String	private	Identifica la sección.
Localidad	precio	double	private	Precio del boleto de la localidad.
Localidad	capacidadMax	int	private	Cantidad máxima de personas en la localidad.
Localidad	boletosVendidos	int	private	Control de disponibilidad.
Ticket	numeroTicket	int	private	Numero aleatorio.
Ticket	numeroA	int	private	Límite inferior.
Ticket	numeroB	int	private	Límite superior.
Ticket	valid	boolean	private	Validación del rango.

Ticket	localidadAsig	Localidad	private	Localidad para el ticket.
GestorVentas	localidades	Localidad[3]	private	Secciones que se administra.
GestorVentas	compradorActual	Comprador	private	Comprador en el programa.
GestorVentas	dineroRecaudado	double	private	Acumulado para el resultado.
Menu	gestor	GestorVentas	private	Referencia al controlador para acciones.
Main	---	---	---	No requiere atributos.

3. Tabla de métodos

Clase	Método	Parámetros	Retorno	Modificador de acceso	Propósito
Comprador	getters/setters	Atributo	Atributo	public	Acceso a los atributos.
Localidad	espacioDisponible()	--	boolean	public	Verifica cupo antes de vender boleto.
Localidad	ventaBoletos(cantidad)	int	int	public	Venta parcial.
Localidad	getDisponibles()	--	int	public	Consultar la disponibilidad.
Localidad	Getter/setter: boletosVendidos	int	int	public	Acceso a atributo boletosVendidos.
Ticket	generarNumeros()	--	void	public	Genera los números de tickets a y b.
Ticket	validarRango()	--	boolean	public	Verifica si el ticket es válido.
Ticket	Getter/Setter: localidadAsig	Localidad	Localidad	public	Acceso a atributo localidadAsig.
GestorVentas	procesarSolicitud(c)	Comprador	String/boolean	public	Procesa toda la compra.

GestorVentas	generarTicket()	--	Ticket	Private	Crea el ticket para proceso de compra.
GestorVentas	localidadAleatoria()	--	Localidad	private	Sorteo de localidad para ticket.
GestorVentas	validarCompra(l,c)	Localidad, Comprador	boolean	private	Validaciones para compra.
GestorVentas	disponibilidadTotal()	--	String	public	Opción de menú.
GestorVentas	disponibilidadIndividual(l)	Localidad	String	public	Opción de menú.
GestorVentas	reporte()	--	double	public	Opción de menú.
Menu	mostrarMenu()	--	void	public	Muestra las opciones.
Menu	ejecutar()	--	Void	public	Ciclo de las interacciones.
Menu	datosComprador()	--	Comprador	private	Lee los datos de un comprador.
Main	Main(args: String[])	String[]	void	public static	Punto principal del programa.

¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)?

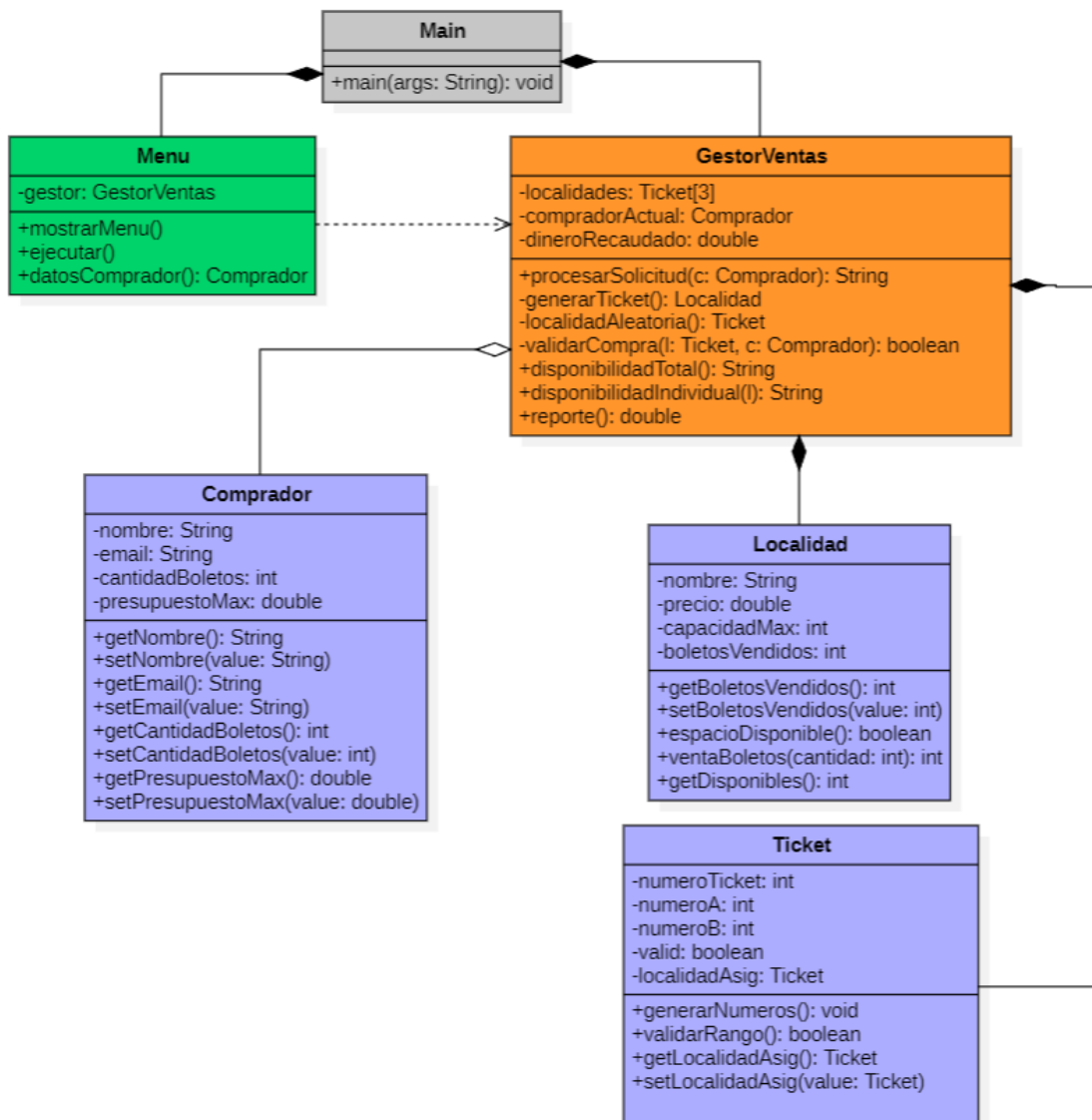
- Model: Estos se encargan de guardar los datos y de ciertas reglas para cumplir con el resto del programa, dentro de esto se encuentran mis clases Comprador, Localidad y Ticket.
- Controller: Estos utilizan los Models para poder realizar el resto de las operaciones solicitadas del programa, en esta carpeta están mi clase GestorVentas.
- View: Se encargan únicamente de leer e imprimir datos, no contienen tanta complejidad, en esta se encuentra mi clase Menu.
- Driver: En esta únicamente se encuentra mi clase Main, el arranque del programa.

¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

En caso de que alguna parte del algoritmo se decida modificar o cambiar la lógica desarrollada con anterioridad como puede ser la asignación de tickets, únicamente se realizan cambios en la clase de GestorVentas y Ticket, no es necesario modificar el resto de las clases con sus archivos. Esto ayuda a poder evitar que un cambio de lógica altere o rompa otra interfaz del usuario o viceversa, ayuda a que cada cambio se realiza a una clase por una razón.

Diseño

Diagrama de clases - UML



Justificaciones de asociaciones

- **Main → Menu / Main → GestorVentas (composición):** Si Main termina, no tiene sentido que Menu o GestorVentas sigan vivos; son creados y destruidos junto con el programa.
- **GestorVentas → Localidad (composición):** Las 3 localidades no existen fuera del sistema de ventas; son parte intrínseca del controlador.
- **GestorVentas → Ticket (composición):** Cada Ticket es temporal, se genera y se consume dentro de procesarSolicitud(); no tiene vida propia fuera de ese proceso.
- **GestorVentas → Comprador (agregación):** El comprador podría existir de forma independiente y solo es "usado" temporalmente por el gestor como comprador activo sin embargo no le pertenece de forma exclusiva.
- **Menu → GestorVentas (dependencia):** Menu solo invoca métodos del controlador, no lo posee ni comparte su ciclo de vida.